

design — let designers & PMs build apps

An auto-triggering Claude Code skill that turns plain-language intent into a working application — no command to type.

Status: Proposal · Owner: dev@potenlab.dev · Date: 2026-06-12 · Review: DionNam, Raka · Timeline: 3 days

The goal. A designer or PM describes an **application** they want — in design and business language — and Claude **creates the whole thing**. It **thinks as an engineer** underneath (data, backend, UI, security) but **responds as a designer/PM**: plain words and a running app, never code. It **switches itself on** from the user's words — **no command to type**.

1 · Why

Today, building even a small internal app needs an engineer's mental model — file trees, diffs, databases, RLS. Designers and PMs have the *ideas* but not that model. The design skill is a **translation skin** over tools we already have: the user just describes what they want, the skill **recognizes the intent and turns itself on**, runs the engineer's loop silently, and shows only the result.

2 · What it does

The user does	The skill does (silently)
Describes an app in plain language	Recognizes the intent & switches on → maps it to app structure + scope
Confirms a plain-language plan	Builds full-stack in an isolated sandbox: data + backend + UI
Looks at before/after screenshots	Runs the app, captures mobile + desktop walkthrough
Says "ship it"	Opens a PR with screenshots + a human summary for the dev

3 · Install once, triggers itself

There is **no command to learn**. Install the skill once and it activates automatically whenever a user describes an app or design — Claude reads the skill's intent triggers and turns it on.

Install scope	Where	Result
Per-user (global)	~/ .claude/skills/	Active on every project that user opens.
Per-project (team)	repo's .claude/skills/	Ships with the project — the whole team gets it.

4 · Building blocks — all ready

Need	Reuse	Status
Intent → structure + blast radius	kg-workflow (kg-context-dispatch)	✓ ready
Plain-language design & UI build	frontend-design, impeccable	✓ ready
Secure backend (data + auth)	supabase-integration skill + Supabase MCP	✓ ready
Run app + visual verify	chrome extension / chrome-devtools MCP, verify	✓ ready
Asset generation	higgsfield MCP	✓ ready
PR handoff	commit-push-pr, qa-check	✓ ready

Every dependency is installed. This is **assembly + guardrails**, not new infrastructure — which is why it fits in 3 days.

5 · Three-day timeline

Day	Ships
Day 1 Triggers & guardrails	~6 golden designer/PM app-idea prompts → acceptance tests. design skill scaffold with a tuned auto-trigger (fires on app/design intent, stays quiet otherwise) → plain-language plan restatement → isolated sandbox per session + hard guardrails.
Day 2 Build & PR	Full-stack build in the sandbox: Supabase backend (schema + RLS + auth + typed client, verified) + UI. Run the app, capture mobile + desktop walkthrough. Auto-PR with summary + screenshots.
Day 3 Dogfood & install	Run the golden prompts with Minjae + a PM set without telling them a command — confirm it auto-fires. Capture friction, tune triggers, document the one-step install (global vs per-project), lock v1.

Usable MVP by end of Day 2.

6 · The guardrail is the product

The skill builds full apps (data + backend included), so safety isn't "never touch the backend" — it's **isolation**. Each app is built in its own sandbox and can never reach our live systems. Because the skill fires **without an explicit command**, this boundary has to hold on its own.

✓ Allowed

- Everything inside the new app's own workspace/branch
- Its data model, backend, UI, assets, config

✗ Hard refuse

- Other apps / repos, production databases
- Shared secrets, existing auth & billing

If an idea genuinely needs a forbidden change, the skill **stops** and writes a dev-handoff note for the engineer — in plain language — instead of doing it.

7 · What the user actually experiences

```
PM: "an intake tool – clients submit a brief, we track it New → Doing → Done"
  ↓ (no command – the skill recognizes the intent and switches on)
design ① restates the plan in plain language
      "A brief tracker. It saves briefs and their status. It lives in its own
      sandbox – nothing touches our live data. Go ahead?"
  ↓ (yes)
      ② builds the full app on an isolated branch (sandbox only)
      ③ runs it, shows BEFORE / AFTER – mobile + desktop
  ↓ (looks good)
      ④ opens a PR with screenshots + a human summary for the dev
```

8 · Done when (v1 acceptance)

- **Auto-triggers on app/design intent** without a command — and stays quiet on unrelated asks.
- Builds a **runnable** app from a plain-language description.
- Stays in its sandbox — refuses to touch live systems, writes a handoff note instead.
- Always restates the plan in plain language before building.
- Always shows the running app (mobile + desktop) before the PR.
- Always lands on an isolated branch and opens a PR — never commits to main.
- Never shows the user raw code unless asked.

9 · We need from the team

- 5–6 real app ideas a designer **and** a PM would actually type (seeds the golden set + tunes the trigger).
- Default install scope — global per-user for everyone, or committed per-project?
- Where new sandbox apps live — a subfolder here, or a fresh repo per app.
- Confirm the isolation boundary (what counts as "production / off-limits").